

Everything you need to build on Rivet

Field-service software for trades contractors. This is the orientation document — what the product is, the one idea the whole codebase is built around, which half of the repository is real, and the rule you cannot get wrong.

AUDIENCE	STAGE	BRANCH	LIVE CODE	REVISED
Engineers	Pre-launch	rebuild/main	~10,400 LOC	2026-08-07

PROBLEM

Quoting speed decides who wins the job.

What Rivet is

A contractor finishes a site visit at 4pm. The quote gets written that evening, or the next morning, or on Monday. By the time it lands, the homeowner has already signed with whoever answered fastest. Trades businesses lose work to latency, not to price.

Rivet collapses that gap. The contractor describes the job in a sentence — *replace 3-ton AC unit, add UV light* — and AI drafts a quote from their own price catalog, with the right tax, in seconds. They adjust it on their phone in the driveway. The customer opens a link, accepts, and pays. The same record then becomes the scheduled job and the invoice.

The market is HVAC, plumbing, electrical, roofing, and landscaping — businesses with two to twenty people, where the owner is still in the field and quoting is overhead taken out of evenings.

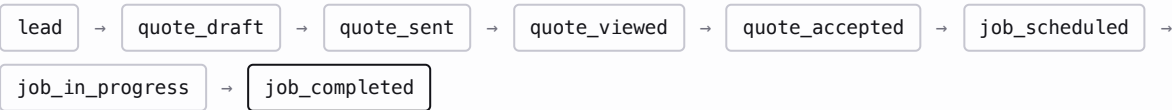
CORE IDEA

One row, not three. Every architectural choice follows from this.

One record, whole lifecycle

Most field-service software models leads, quotes, and jobs as separate tables and copies data between them. Rivet doesn't. A single `work_items` row *is* the lead, then the quote, then the job, then the archived record. Its `status` column moves; the row never does.

PRIMARY PATH



EXITS



Converting a lead to a quote is `UPDATE status = 'quote_draft'`. No row copy. That one property buys a lot: the link you already sent the customer keeps working, the activity log stays continuous, ids stay stable for the audit trail, and analytics is one `GROUP BY` instead of a three-way union.

The cost is a wide table — roughly forty columns, many meaningful at only one stage. `scheduled_start` means nothing on a lead; `sent_at` means nothing on a job. That's an accepted trade, and a generated `kind` column (`lead | quote | job | archived`) keeps board filters and indexes cheap.

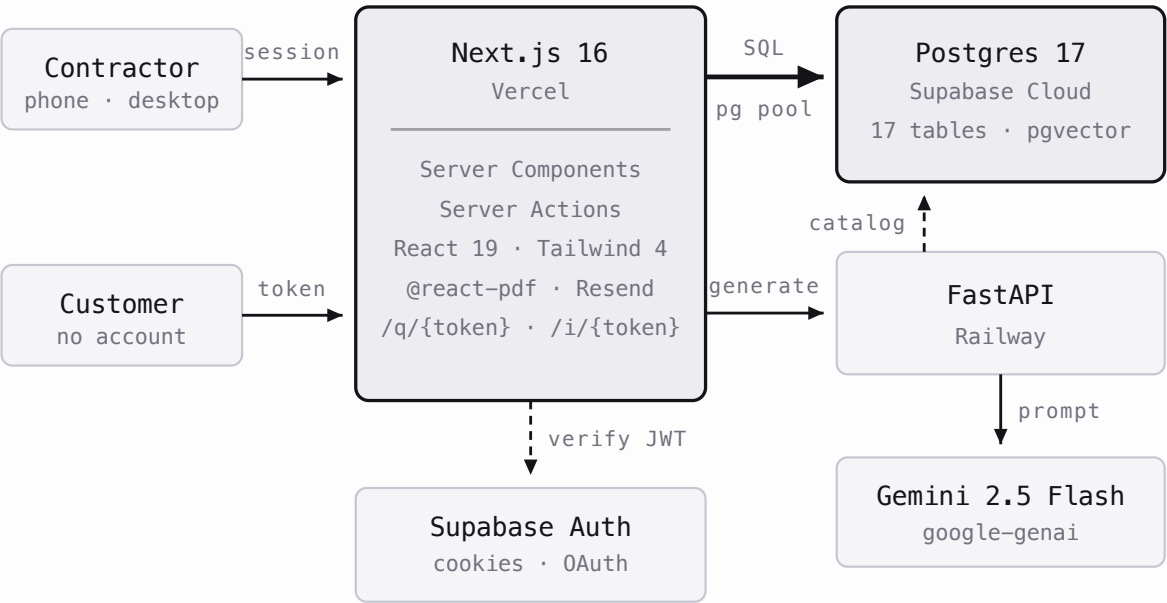
Around it sit `companies` (the tenant root), `users`, `customers`, `catalog_items` (the contractor's price book), `quote_items`, `invoices`, and `payments`. Seventeen tables in total.

STACK

Two runtimes,
one database.
Everything else
is a library or
a vendor.

The stack

Two deployable units: a Next.js application and a small Python AI service. One Postgres database behind both. Nothing else runs.



The load-bearing edge is the thick one. Reads and writes go straight from the Next.js server to Postgres over a connection pool — there is no API tier between them. Supabase is contacted only to verify identity (dashed). The AI service is a side call on the quote path, not in the request path for anything else.

LAYER	CHOICE	STATE	NOTE
Framework	Next.js 16	IN USE	App Router, React Compiler on, security headers in <code>next.config.ts</code>
UI	React 19 · Tailwind 4 · shadcn/ui	IN USE	19 primitives in <code>src/components/ui</code> ; monochrome tokens in <code>globals.css</code>

LAYER	CHOICE	STATE	NOTE
Language	TypeScript strict	IN USE	Build errors ignored only because the dead tree doesn't compile
Validation	Zod · React Hook Form	IN USE	Also validates env vars and the AI service response contract
Database	Postgres 17	IN USE	Via node-postgres, parameterized SQL, no ORM
Auth	Supabase Auth	IN USE	@supabase/ssr cookies, Google OAuth. Identity only, never data.
AI	FastAPI · google-genai · Gemini	IN USE	Grounded on the catalog, model fallback chain, mock fallback
PDF	@react-pdf/renderer	IN USE	Server-rendered quote and invoice documents
Email	Resend · React Email	IN USE	Templates in src/emails. Domain auth not yet configured.
Payments	Stripe Connect Express	TEST MODE	Checkout and card-fee pass-through wired; live keys pending
E-signature	SignNow / Dropbox Sign	TEST MODE	Falls back to instant acceptance if signing fails
Retrieval	pgvector · tsvector	NOT WIRED	Schema and match_documents() exist; nothing calls them
Billing	LemonSqueezy	NOT WIRED	How Rivet itself would be charged for. Does not exist yet.
Hosting	Vercel · Railway · Supabase	DECIDED	Not provisioned. Nothing is deployed anywhere today.
Tooling	pnpm · biome · vitest · uv · ruff	TARGET	Repo currently runs npm + eslint with no test runner

DATA FLOW

One quote, front to back. Six hops, two runtimes.

How a quote moves through the system

This is the path worth tracing on your first day. It touches the data layer, tenancy, server actions, the AI service, and the public surface — once you can narrate it, you can work anywhere in the codebase.



Status changes, the row doesn't. Steps 3 through 6 all act on the same `work_items` row: `quote_draft` → `quote_sent` → `quote_viewed` → `quote_accepted`. Scheduling and invoicing continue on that same row, which is why the link in step 6 keeps working for the life of the job.

RATIONALE

Why the code looks the way it does.

Three choices that explain the code

Postgres directly, not through an ORM

Every read and write goes through a `pg` pool and parameterized SQL — `src/lib/db/index.ts`. No Drizzle, no Prisma, no Supabase query builder.

Supabase is still the database host and still owns authentication. It is not the data layer. PostgREST forced awkward shapes on multi-table reads — embeds that break silently when a column is renamed, which caused two production 404s — and joins were being emulated in TypeScript. Plain SQL also keeps the door open to move off Supabase without rewriting the application.

Two things in that file surprise people: `numeric` is parsed to a JS `number` so money arithmetic works, and timestamps come back as raw ISO strings rather than `Date` objects.

Server Actions for writes, Server Components for reads

There is no REST API for the product. A route directory owns its own data access: a `page.tsx` that queries and renders, an `actions.ts` holding `'use server'` mutations, and client components only where there's interaction.

Actions never throw to the client. They validate with Zod, verify ownership, mutate, revalidate the affected paths, and return `{ ok: true, data } | { ok: false, error }`. Errors surface as toasts, not error boundaries. A page render is one server-side round trip to Postgres — no client fetch layer, no cache to invalidate, no API schema to keep in sync.

Public links are tokens, not ids

`/q/{token}` and `/i/{token}` are the only unauthenticated routes that read data. The token is 128 random bits on the work item, never its UUID, so links can't be enumerated. These routes use the service-role client because there is no session to scope by — the token *is* the authorisation. That's a deliberate exception, not an oversight.

It is also the surface customers judge the product on, often the only screen they ever see. It should feel like Stripe Checkout.

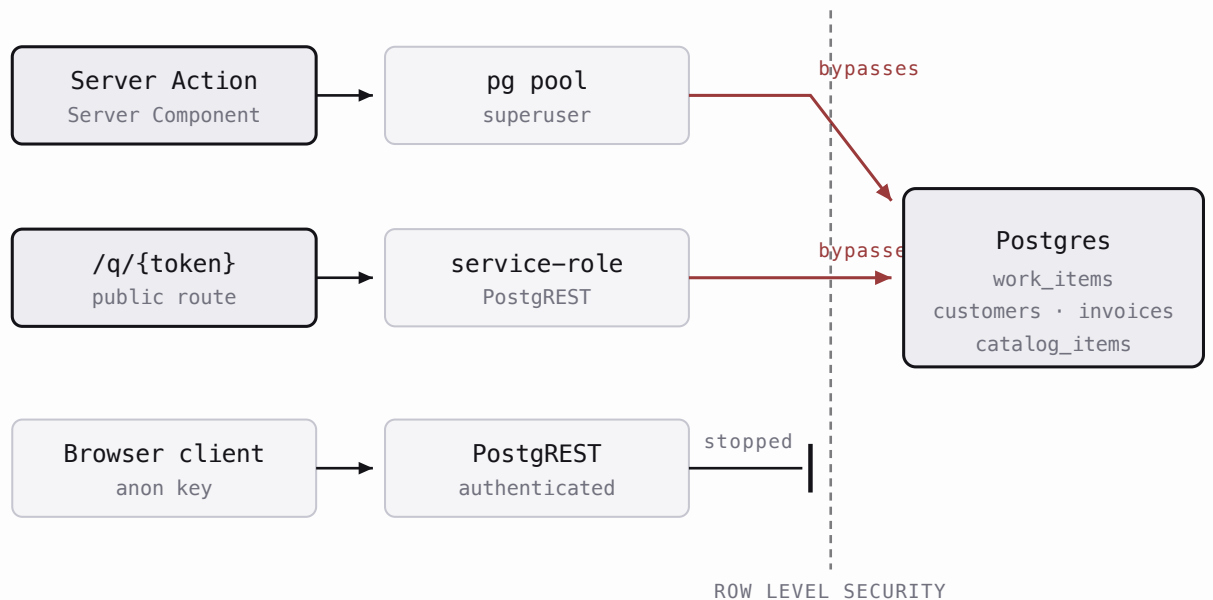
CRITICAL

No framework catches a missed tenant filter. Nothing does.

The rule you cannot get wrong

The `pg` pool connects as superuser and bypasses Row Level Security.

RLS policies exist on every table and are correct — but they protect the `anon` and `authenticated` Postgres roles, not your queries. Application-level scoping is the primary control, and it is manual.



Both application paths go around the policy layer. For server code the real boundary is the `where company_id = $n` clause you write; for the public route it is the unguessable 128-bit token. Row Level Security only stops the path the application doesn't use.

So: every statement touching company data carries `company_id`, and every mutation verifies the target row belongs to the caller's company before it writes. A missing clause is a cross-tenant data leak that compiles, passes review, and looks fine in local testing against a single seeded company.

```

// Right
const { companyId } = await requireSession()
const rows = await query<Row>(
  'select id, total from work_items where id = $1 and company_id = $2',
  [id, companyId],
)

// Wrong – returns another contractor's quote
const rows = await query<Row>(
  'select id, total from work_items where id = $1',
  [id],
)
  
```

Two session helpers exist, and the distinction matters: `requireSession()` redirects and belongs in pages; `getSession()` returns null and belongs in actions. Both verify the JWT through Supabase, then read the user row via `pg`.

One more trap: several SQL functions read `auth.uid()` internally. Under the raw pool that's NULL. Those calls go inside `withUser(userId, ...)`, which sets the JWT claims inside the transaction first.

Roughly half the repository is unreachable pre-rebuild code that still compiles.

Which half is real

Rivet was rebuilt in place. The previous application was never deleted — it still compiles, still imports the old Supabase data layer, and still turns up in every grep. Editing it is the most common way to waste a day here.

The classification below was derived by tracing every route reachable from the root page and middleware, then computing the transitive import closure. Anything outside it is dead.

PATH	STATE	NOTES
src/app/app/**	LIVE	The entire signed-in product — dashboard, pipeline, quotes, calendar, customers, catalog, analytics, settings
src/app/{q,i,join,login,auth}/**	LIVE	Public quote and invoice viewers, invitations, auth
src/lib/{db,auth,email,pdf,stripe}	LIVE	Data layer, session helpers, Resend, PDF rendering, Connect
src/components/{ui,brand,shared}	LIVE	19 shadcn primitives, logo system, page scaffolding
python-backend/ai_backend.py	LIVE	340 lines. The only Python file that runs.
src/app/(dashboard)/**	DEAD	Pre-rebuild UI. Unreachable. The sole reason strict builds are disabled.
src/app/api/**	DEAD	Old REST layer, replaced by Server Actions. Except /api/vitals.
src/components/{features,queues,navigation,calendar,ai}	DEAD	Old UI systems, including duplicate component pairs
src/hooks/**, src/features/{work-items,catalog,ai}	DEAD	React Query hooks and rebuild-era code that was never wired up
python-backend/{src,app,api,services}	DEAD	Three further backend generations. See the note below.
supabase/migrations/legacy/**	DEAD	Includes EMERGENCY_DISABLE_RLS.sql. Never run these.

One consequence worth stating plainly: `next.config.ts` sets `typescript: { ignoreBuildErrors: true }` purely because the dead tree doesn't compile. That is not permission

to ship type errors — `tsc --noEmit` is the real gate, and deleting the dead tree is what lets the flag come off.

The most interesting dead code is `python-backend/src/quotepror/` : a complete, tested FastAPI application with multi-agent routing, hybrid RAG over pgvector, Postgres-backed sessions, and a background indexer — built during the rebuild and never connected. Whether to adopt or delete it is an open decision.

MODEL POLICY

Google Gemini only. Standing decision, not a default.

How the AI works

A single FastAPI endpoint. It fetches the company's active catalog items, builds a grounded prompt, and calls Gemini through the unified `google-genai` SDK — the same client targets AI Studio via API key or Vertex AI via application credentials.

Grounding is the whole game. The system prompt forbids inventing line items. That constraint is load-bearing: a hallucinated price is one the contractor is contractually on the hook for once the customer accepts. Quote quality is therefore a function of catalog quality, not model cleverness — which is why catalog import and onboarding matter more to output than any prompt tweak.

Two fallbacks, both deliberate. The model chain is tried in order, so a quota limit on the newest Flash model degrades instead of failing. If every model fails, a keyword matcher over the catalog returns plausible line items and flags the response as `mock` — the UI stays fully exercisable offline, and a Gemini outage doesn't take quoting down. Production should alert whenever that flag appears.

Settings are fixed: temperature at or below 0.2, JSON response type whenever output is parsed, response schema where the SDK supports one. Money and structured output must be deterministic. Prompts live as markdown in `prompts/`, not as string literals in Python, so behaviour can be edited and diffed without touching code.

OPEN RISK The AI service currently has no authentication and permits all origins.

It accepts `company_id` in the request body and returns that company's catalog-derived pricing. Deployed publicly as-is, anyone who finds the URL can read any tenant's pricing and exhaust the Gemini quota. Fixing this — verify the JWT, derive `company_id` from the token, lock CORS, rate-limit — blocks launch.

HONEST STATUS

Works end to end locally. Not deployed anywhere.

Where it actually stands

The application works. You can sign up, build a catalog, generate a quote, send it, watch a customer accept it, schedule the job, invoice it, and record payment. That whole loop runs today — against a local database, on a laptop.

Product	Complete quote-to-cash loop. Dashboard, pipeline board, calendar, customers, catalog, analytics, team invitations with role personas.
Infrastructure	None. Local Supabase behind disposable Cloudflare tunnels. No production database, no deployment, no domain.
Payments	Stripe Connect wired, test mode. Rivet's own billing — plans, checkout, entitlement gating — does not exist yet.
Email	Resend wired. Domain authentication (SPF, DKIM, DMARC) not configured, so production quotes would land in spam.
Tests	None. <code>tsc --noEmit</code> is the only automated check. CI describes a toolchain that isn't installed.
Front door	No landing page. The root path redirects straight to login.

The sequenced path to production runs roughly six weeks: hosted infrastructure and dead code removal in parallel, then security and correctness, then Stripe live mode and billing, then the landing page and observability. Deleting the dead tree and standing up hosted Supabase are complementary — the first is what lets a strict production build pass, the second is what lets more than one person work on it.

SETUP

Two required processes.
Third only for AI.

Getting productive

Clone somewhere local first. The repository currently lives in iCloud Drive, which syncs every file operation — installs crawl and git history walks can hang for minutes.

```
git switch rebuild/main # the live branch; `main` is pre-rebuild
npm install
supabase start          # Postgres :54322 · API :54321 · Studio :54323
supabase db reset       # migrations + demo seed
npm run dev             # → localhost:3000

# quote generation only:
cd python-backend && uvicorn ai_backend:app --reload --port 8000
```

The seed creates one demo company with fifteen work items spread across the lifecycle, and three logins — owner, office, and technician, all `demo1234`. Test with more than one. Permission bugs only appear when you switch roles.

Two things will bite you on first run. `node_modules` may be a symlink pointing at a directory that doesn't exist — an iCloud workaround; delete it and reinstall. And the inherited environment file points at expired tunnel URLs; replace them with localhost.

Then trace one flow

Create a quote in the UI and follow it through the code, hop by hop, against the six-step diagram above. Open the pipeline detail page, then pull the `public_token` out of the database and open `/q/<token>` in a private window — that's the screen your customer sees, and it's the one that sells.

Working agreement

- Branch off `rebuild/main` . Small PRs, one concern each.
- `tsc --noEmit` clean on live code before opening.
- Terse commits — `fix(pipeline): detail 404 on missing address` . No summary paragraphs.
- No new markdown files unless asked. The repo has roughly 140 and most are misleading.
- Non-obvious decisions get a one-page ADR: context, decision, consequences, date.

AGENT SETUP

Fifteen minutes. Do it before your first real task.

Setting up Claude Code

The project context, the coding rules, and the repeatable workflows are committed to the repository rather than living in anyone's head or chat history. A clone brings them with it, so no one has to re-explain Rivet to an agent.

CLAUDE.md	Loads automatically. Product summary, live/dead code map, tenancy rule, commands, traps.
.claude/skills/	Six project skills — <code>rivet-dev</code> , <code>rivet-data</code> , <code>rivet-migration</code> , <code>rivet-ui</code> , <code>rivet-ai</code> , <code>rivet-ship</code> .
.claude/settings.json	The team's plugin set, so every machine has the same tooling. Committed.
.claude/settings.local.json	Your personal permission allowlist. Gitignored.

The steps

```
# 1 · install
curl -fsSL https://claude.ai/install.sh | bash

# 2 · authenticate (first run walks you through sign-in)
claude

# 3 · clone OUTSIDE iCloud Drive, onto the live branch
git clone https://github.com/dipenvekaria/quotepron.git ~/code/rivet
cd ~/code/rivet && git switch rebuild/main

# 4 · start Claude FROM THE REPO ROOT – this is the step people get wrong
claude
```

Then confirm it worked. `/context` should list `CLAUDE.md` among the loaded files, and `/rivet-dev` should resolve. If the skills aren't found, you started Claude from the wrong directory — `CLAUDE.md` loads from the working directory and its parents, so launching from a parent silently gives you an agent with no project context at all.

The real test is asking something only the repo context can answer — *which parts of this codebase are dead?* A correctly configured session names the `(dashboard)` tree, most of `src/app/api`, and three of the four Python backends. One that starts grepping around is not loading the context.

Staying in sync

Because the files are the source of truth, keeping everyone aligned is a matter of updating them rather than telling each other things. If Claude gets something wrong about the project and you correct it in chat, that correction dies with the session — put it in `CLAUDE.md` or the relevant skill and commit it. A new repeatable workflow becomes a new skill under `.claude/skills/`, and it's on everyone's machine at their next pull.

Of the six skills, `rivet-data` is the one that earns its place most. The `pg` pool bypasses row level security, so tenant scoping is enforced by hand in every query — an agent that hasn't read that skill will write a plausible-looking query that leaks another contractor's data.

Where the detail lives

Everything above is the orientation layer. The repository carries the full versions, and they are the ones kept current.

CLAUDE.md	Agent context — rules, live/dead map, conventions. Loads automatically in Claude Code.
docs/ONBOARDING.md	Day one through first PR, with the failure modes spelled out.

REFERENCE
In the repo.
Written 2026-08-07 and verified against the working tree.

docs/CLAUDE_CODE_SETUP.md	Full agent setup, plugin set, and how the team stays in sync.
docs/CODEBASE_MAP.md	Every directory classified live or dead, with reasoning.
docs/ARCHITECTURE.md	How the pieces fit and why each choice was made.
docs/DATA_MODEL.md	Schema, lifecycle, views, functions, RLS.
docs/CONVENTIONS.md	Formatting, naming, data access, server actions, comments, git.
docs/DEPLOYMENT.md	Vercel, Railway, Supabase Cloud — the runbook, not a description.
docs/LAUNCH_PLAN.md	Sequenced path to production, P0 through P2.
docs/CLEANUP_PLAN.md	Debt paydown in dependency order.
docs/adr/	Decisions and their rationale, including the ones taken this week.

Six project skills live in `.claude/skills/` and give Claude Code the right instincts here: `rivet-dev` for running the stack, `rivet-data` for queries and tenancy, `rivet-migration` for schema changes, `rivet-ui` for the design system, `rivet-ai` for the model layer, and `rivet-ship` for pre-PR verification.

Anything under `docs/archive/`, `docs/rebuild/`, or `REBUILD.md` describes a system that no longer exists. History, not instruction.